

GUIADE ENTREVISTA

Desenvolvedor de IA Júnior

Banco Bari

Guia Completo com Perguntas e Respostas

Python • SQL • Estruturas de Dados • POO • SOLID Clean Code • RAG • LLMs
• AWS • Comportamentais

Renzo — Hermes Agent
Junho 2026

■ INTRODUÇÃO — Como Usar Este Guia

Este guia foi montado especificamente para a vaga de **Desenvolvedor de IA Júnior** no Banco Bari. A seleção pode conter fases de: triagem de currículo, prova técnica (ao vivo ou com tempo limitado), entrevista com o time de tecnologia e entrevista com RH/gestão.

Cada seção abaixo contém as perguntas mais frequentes em cada tema, com respostas detalhadas e exemplos de código sempre que possível. As respostas são completas o suficiente para você demonstrar profundidade técnica — não apenas decorar.

■ ATENÇÃO — A vaga pede Sênior

Originalmente a vaga era Sênior, mas você se candidatou para júnior. Isso significa que o nível de profundidade esperado nas respostas pode ser um pouco mais alto do que o normal para júniores — estão testando se você consegue operar no nível sênior. Foque em demonstrar: (1) lógica sólida, (2) projetos pessoais ou estudos profundos em IA, (3) capacidade de explicar conceitos com clareza.

■ Estrutura Típica da Entrevista no Bari:

- 1■■ Triagem RH (30 min) — comportamento, cultura, motivação
- 2■■ Prova Técnica (60-90 min) — Python + SQL + lógica (ao vivo ou assíncrono)
- 3■■ Entrevista com time de IA/engenharia (45-60 min) — RAG, LLMs, arquitetura, projetos
- 4■■ Entrevista final com gestão (30 min) — alinhamento cultural,■■

■ SEÇÃO 1 — PYTHON

Python é a linguagem central da vaga. Você precisa dominar desde fundamentos até conceitos intermediários (decorators, generators, OOP). A prova técnica geralmente inclui problemas de lógica em Python.

1.1 Listas e List Comprehension

■ O que é list comprehension? Dê um exemplo prático com filtro e transformação.

```
# Exemplo 1: quadrados dos números pares de 0 a 9
quadrados_pares = [x**2 for x in range(10) if x % 2 == 0]
print(quadrados_pares) # [0, 4, 16, 36, 64]
# Exemplo 2: filtrar nomes que começam com 'A'
nomes = ['Ana', 'Bruno', 'Amanda', 'Carlos', 'André']
com_a = [n for n in nomes if n.startswith('A')]
print(com_a) # ['Ana', 'Amanda', 'André']
# Exemplo 3: matriz transposta (list comprehension aninhada)
matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
transposta = [[linha[i] for linha in matriz] for i in range(3)]
print(transposta) # [[1, 4, 7], [2, 5, 8], [3, 6, 9]]
# Exemplo 4: dicionário a partir de listas
keys = ['nome', 'idade', 'cidade']
values = ['Ana', 28, 'Curitiba']
dict_comp = {k: v for k, v in zip(keys, values)}
print(dict_comp) # {'nome': 'Ana', 'idade': 28, 'cidade': 'Curitiba'}
```

List comprehension é uma forma concisa de criar listas em Python usando uma sintaxe declarativa. Ela substitui loops for tradicionais e é mais legível e rápida. A estrutura é: **[expressão for item in iterable if condição]**

■ Qual a diferença entre append() e extend() em listas?

```
a = [1, 2, 3]
a.append([4, 5])
print(a) # [1, 2, 3, [4, 5]] ← a lista interna é UM elemento
a = [1, 2, 3]
a.extend([4, 5])
print(a) # [1, 2, 3, 4, 5] ← cada inteiro é um elemento separado
# append é útil para pilhas (stack)
pilha = []
pilha.append(1) # [1]
pilha.append(2) # [1, 2]
topo = pilha.pop() # [1], topo = 2
```

`append()` adiciona um único item ao final da lista (como elemento único, inclusive outra lista). `extend()` adiciona múltiplos itens ao final, iterando sobre o iterável fornecido (desempacotando cada item). A diferença prática: `append()` aumenta o tamanho da lista em 1; `extend()` adiciona cada item do iterável individualmente.

■ Como funciona tuple unpacking? Dê exemplos avançados.

```
# Desempacotamento básico
x, y, z = (1, 2, 3)
print(x, y, z) # 1 2 3
# Trocar valores sem variável temporária
a, b = 10, 20
a, b = b, a
# a = 20, b = 10 – padrão comum em algoritmos
# Usar * para capturar o resto
head, *body, tail = [1, 2, 3, 4, 5]
print(head) # 1
print(body) # [2, 3, 4]
print(tail) # 5
# Desempacotamento aninhado
(a, (b, c)) = (1, (2, 3))
print(a, b, c) # 1 2 3
# Em funções que retornam múltiplos valores
def get_estatisticas(numeros):
    return min(numeros), max(numeros), sum(numeros)/len(numeros)
mn, mx, avg = get_estatisticas([10, 20, 30, 40])
print(f'Min: {mn}, Max: {mx}, Avg: {avg:.1f}') # Min: 10, Max: 40, Avg: 25.0
# Extended unpacking com strings
primeiro, *resto = 'Python'
print(primeiro, resto) # P ['y', 't', 'h', 'o', 'n']
```

Tuple unpacking (ou desempacotamento) permite extrair valores de uma tupla/lista em variáveis separadas. É uma das features mais elegantes do Python.

1.2 Dicionários

■ Como criar e manipular dicionários de forma eficiente? O que é defaultdict?

```
from collections import defaultdict
# defaultdict – especifique o tipo padrão
dd = defaultdict(int) # default: 0 para inteiros
dd['vendas'] += 1 # não dá erro se 'vendas' não existir
print(dd) # defaultdict(<int object at 0x...>, {'vendas': 1})
# defaultdict com list
dd_list = defaultdict(list)
dd_list['python'].append('Django')
dd_list['python'].append('FastAPI')
```

```
dd_list['java'].append('Spring') print(dd_list) # {'python': ['Django', 'FastAPI'], 'java': ['Spring']} # defaultdict com lambda – exemplo: words = ['apple', 'banana', 'apricot', 'blueberry', 'cherry'] group_by_first = defaultdict(list) for word in words: group_by_first[word[0]].append(word) print(dict(group_by_first)) # {'a': ['apple', 'apricot'], 'b': ['banana', 'blueberry'], 'c': ['cherry']} # Dict comprehension com condições dados = [('nome', 'Ana'), ('idade', 28), ('cidade', 'Curitiba'), ('nome', 'Bruno')] unique = {k: v for k, v in dados} # última ocorrência sobrescreve print(unique) # {'nome': 'Bruno', 'idade': 28, 'cidade': 'Curitiba'} # merge de dicionários (Python 3.9+) d1 = {'a': 1, 'b': 2} d2 = {'b': 3, 'c': 4} merged = d1 | d2 print(merged) # {'a': 1, 'b': 3, 'c': 4} ← b foi sobrescrito pelo d2
```

Dicionários em Python são tabelas hash — acesso O(1) em média. O defaultdict é um dicionário do módulo collections que retorna um valor padrão para chaves inexistentes, evitando KeyError.

■ Qual a diferença entre dict e OrderedDict?

```
# Python 3.7+ – dict mantém ordem de inserção d = {} d['z'] = 1 d['a'] = 2 d['m'] = 3
print(list(d.keys())) # ['z', 'a', 'm'] – ordem garantida from collections import OrderedDict #
OrderedDict tem métodos extras od = OrderedDict() od['z'] = 1 od['a'] = 2 od['m'] = 3
od.move_to_end('a') # move 'a' para o final print(list(od.keys())) # ['z', 'm', 'a'] # Em Python
3.9+, dict também suporta | para merge # e dict.copy() é shallow copy padrão
```

Em Python 3.7+, dicionários comuns mantêm a ordem de inserção por padrão — já se comportam como OrderedDict. O OrderedDict ainda existe para compatibilidade e para funcionalidades específicas como move_to_end().

1.3 Funções: args, kwargs, decorators, lambda

■ Explique *args e **kwargs. Quando usar cada um?

```
def func_exemplo(*args, **kwargs): print(f'Positional args: {args}') print(f'Keyword args:
{kwargs}') func_exemplo(1, 2, 3, nome='Ana', idade=28) # Positional args: (1, 2, 3) # Keyword
args: {'nome': 'Ana', 'idade': 28} # Reaproveitando argumentos com * e ** def saudar(nome,
saudacao='Olá'): print(f'{saudacao}, {nome}!') dados = ('Ana', 'Bom dia') saudar(*dados) #
desempacota tuple → nome='Ana', saudacao='Bom dia' config = {'saudacao': 'Boa noite', 'nome':
'Bruno'} saudar(**config) # desempacota dict → nome='Bruno', saudacao='Boa noite' # Ordem:
parâmetros fixos → *args → kwargs def ordem(a, b, *args, debug=False, **kwargs): print(f'a={a},
b={b}, args={args}, debug={debug}, kwargs={kwargs}') ordem(1, 2, 3, 4, debug=True,
cidade='Curitiba') # a=1, b=2, args=(3, 4), debug=True, kwargs={'cidade': 'Curitiba'}
```

*args permite que uma função receba múltiplos argumentos posicionais, que são agrupados em uma tupla.

**kwargs permite que uma função receba múltiplos argumentos nomeados (key=value), que são agrupados em um dicionário. O nome 'args' e 'kwargs' é convenção — o importante é o *, **.

■ O que é um decorator? Implemente um que mede tempo de execução e um com parâmetros.

```
# DECORATOR BÁSICO – mede tempo de execução import time def timer_decorator(func): def
wrapper(*args, **kwargs): inicio = time.time() resultado = func(*args, **kwargs) duracao =
time.time() - inicio print(f'■ {func.__name__} executou em {duracao:.4f}s') return resultado
return wrapper @timer_decorator def processar_dados(lista): return sum(x**2 for x in lista) result
= processar_dados(range(10000)) # ■ processar_dados executou em 0.0008734s # DECORATOR COM
PARÂMETROS def repeat(times): def decorator(func): def wrapper(*args, **kwargs): resultados = []
for _ in range(times): resultados.append(func(*args, **kwargs)) return resultados return wrapper
return decorator @repeat(times=3) def saudar(nome): return f'Olá, {nome}!' print(saudar('Ana')) #
['Olá, Ana!', 'Olá, Ana!', 'Olá, Ana!'] # DECORATOR PARA VALIDAÇÃO DE INPUT def
validar_positivo(func): def wrapper(*args, **kwargs): for arg in args: if isinstance(arg, (int,
float)) and arg < 0: raise ValueError(f'Valor negativo não permitido: {arg}') return func(*args,
**kwargs) return wrapper @validar_positivo def calcular_raiz(x): return x ** 0.5
```

```
print(calcular_raiz(16)) # 4.0 # calcular_raiz(-16) # ValueError: Valor negativo não permitido:
-16 # Múltiplos decorators – ordem de execução: bottom-up @timer_decorator @validar_positivo def
dobrar(x): return x * 2 # primeiro validar_positivo → depois timer_decorator print(dobrar(5))
```

Decorator é uma função que recebe outra função como argumento e retorna uma nova função com comportamento modificado. É a forma Pythonica de implementar o padrão Decorator do GoF.

■ O que é lambda? Quando usar e quando NÃO usar?

```
# Lambda básico soma = lambda a, b: a + b print(soma(3, 4)) # 7 # Uso prático: key function em
sorted pessoas = [{'nome': 'Ana', 'idade': 28}, {'nome': 'Bruno', 'idade': 22}] sorted_by_idade =
sorted(pessoas, key=lambda p: p['idade']) print(sorted_by_idade) # [{'nome': 'Bruno', 'idade':
22}, {'nome': 'Ana', 'idade': 28}] # Uso prático: filter + map numeros = [1, 2, 3, 4, 5, 6, 7, 8,
9, 10] pares_quadrados = list(map(lambda x: x**2, filter(lambda x: x % 2 == 0, numeros)))
print(pares_quadrados) # [4, 16, 36, 64, 100] # functools.reduce from functools import reduce
produto = reduce(lambda a, b: a * b, [1, 2, 3, 4]) print(produto) # 24 # ■ QUANDO NÃO USAR –
código ilegível result = list(map(lambda x: x**2 if x > 0 else 0, filter(lambda x: x % 2 == 0,
data))) # ↑ Prefira compreensão de lista result = [x**2 for x in data if x > 0 and x % 2 == 0] # ■
NÃO USE lambda para atribuir a uma variável (defina com def) square = lambda x: x**2 # não faça
isso def square(x): return x**2 # faça isso
```

Lambda é uma função anônima (sem nome) definida em uma única expressão. Útil para funções pequenas e descartáveis. **Não usar** para funções complexas que precisam de múltiplas linhas, documentação ou reutilização.

1.4 Generators e Iterators

■ O que é um generator? Qual a vantagem sobre listas? Implemente um pipeline de dados.

```
# GENERATOR básico –yield def contador(max): n = 0 while n < max: yield n # pausa a função,
devolve valor n += 1 for i in contador(5): print(i) # 0, 1, 2, 3, 4 – memória O(1), não
[0,1,2,3,4] # COMPARAÇÃO: lista vs generator import sys lista_grande = list(range(1000000))
generator_grande = (x for x in range(1000000)) print(sys.getsizeof(lista_grande)) # ~8 MB (toda a
lista em memória) print(sys.getsizeof(generator_grande)) # ~200 bytes (só o objeto generator) #
PIPELINE DE DADOS com generators def ler_linhas(arquivo): """Generator que lê arquivo linha por
linha sem carregar tudo.""" with open(arquivo, 'r') as f: for linha in f: yield linha.strip() def
filtrar_vendas(linhas): """Filtra apenas linhas de vendas.""" for linha in linhas: if
linha.startswith('VENDA'): yield linha def extrair_valor(linhas): """Extraí o valor monetário."""
for linha in linhas: partes = linha.split(',') if len(partes) >= 3: yield float(partes[-1]) def
pipeline_vendas(arquivo): return extrair_valor(filtrar_vendas(ler_linhas(arquivo))) # Uso:
processa GB de dados sem estourar memória total = sum(pipeline_vendas('vendas_2025.csv'))
print(f'Total de vendas: R$ {total:,.2f}') # Generator expression (similar a list comprehension,
mas não carrega em memória) quadrados = (x**2 for x in range(1000000)) # não aloca 1M de ints
print(type(quadrados)) #
```

Generator é uma função que usa `yield` para devolver valores um a um, sem armazenar tudo em memória. Vantagem: memória O(1) vs O(n) de listas. Perfeito para processar streams grandes de dados.

■ O que é itertools? Dê exemplos práticos.

```
import itertools # count – gerador infinito de números for i, nome in zip(range(5),
itertools.count(start=1)): print(f'#{i+1}') # #1, #2, #3, #4, #5 # cycle – repete infinitamente
ciclador = itertools.cycle(['A', 'B', 'C']) for _ in range(8): print(next(ciclador)) # A B C A B C
A B # accumulate – soma acumulada nums = [1, 2, 3, 4, 5] acumulado =
list(itertools.accumulate(nums)) print(acumulado) # [1, 3, 6, 10, 15] # permutations – todas as
ordens possíveis letters = ['A', 'B', 'C'] for perm in itertools.permutations(letters, 2):
print(perm, end=' ') # ('A', 'B') ('A', 'C') ('B', 'A') ('B', 'C') ('C', 'A') ('C', 'B') print() #
combinations – grupos sem repetição for comb in itertools.combinations(['D', 'E', 'F'], 2):
```

```
print(comb, end=' ') # ('D','E') ('D','F') ('E','F') print() # product – produto cartesiano cores
= ['preto', 'branco'] tamanhos = ['P', 'M', 'G'] for item in itertools.product(cores, tamanhos):
print(item, end=' ') # ('preto','P') ('preto','M') ... print() # groupby – agrupar elementos
consecutivos data = [(' animal', 'gato'), (' animal', 'cachorro'), (' planta', 'flor')] for chave,
grupo in itertools.groupby(data, key=lambda x: x[0]): print(f'{chave.strip()}: {list(grupo)}')
```

itertools é um módulo que fornece ferramentas para criação de iterators eficientes. Contém funções para loops, combinações, permutações e ciclos infinitos.

1.5 Programação Orientada a Objetos (POO)

■ Explique os 4 pilares da POO em Python com exemplos de código.

```
# ===== # 1. ENCAPSULAMENTO – ocultar
detalhes internos # ===== class
ContaBancaria: def __init__(self, saldo_inicial): self.__saldo = saldo_inicial # __ = private
(name mangling) self.__historico = [] #getter e setter @property def saldo(self): return
self.__saldo @saldo.setter def saldo(self, valor): if valor < 0: raise ValueError('Saldo não pode
ser negativo') self.__saldo = valor def depositar(self, valor): if valor <= 0: raise
ValueError('Valor deve ser positivo') self.__saldo += valor self.__historico.append(f'+{valor}')
return self def sacar(self, valor): if valor > self.__saldo: raise ValueError('Saldo
insuficiente') self.__saldo -= valor self.__historico.append(f'-{valor}') return self def
extrato(self): return self.__historico.copy() # retorna cópia, não referência #
===== # 2. HERANÇA – reuse e
extensibilidade # ===== class Funcionario:
def __init__(self, nome, salario): self.nome = nome self.salario = salario def trabalhar(self):
return f'{self.nome} está trabalhando.' def calcular_bonus(self): return self.salario * 0.1 #
Herança simples class Desenvolvedor(Funcionario): def __init__(self, nome, salario, linguagem):
super().__init__(nome, salario) # chama __init__ do pai self.linguagem = linguagem def
calcular_bonus(self): # Sobrescrita (polimorfismo) return self.salario * 0.15 # 15% para devs def
trabalhar(self): return f'{self.nome} está programando em {self.linguagem}.' # Herança múltipla –
problema do diamante class Vampiro: def __init__(self, nome): self.nome = nome def morder(self):
return f'{self.nome} mordeu alguém!' class Atleta: def __init__(self, nome): self.nome = nome def
correr(self): return f'{self.nome} está correndo.' #Python não tem problema do diamante por MRO
(Method Resolution Order) class SuperHumano(Vampiro, Atleta): pass sh = SuperHumano('Ana')
print(sh.morder()) # Ana mordeu alguém! print(sh.correr()) # Ana está correndo.
print(SuperHumano.__mro__) # ordem de resolução de métodos #
===== # 3. POLIMORFISMO – mesma interface,
comportamentos diferentes # ===== class
Ave: def voar(self): return 'voando...' class Pinguim(Ave): def voar(self): # sobrescreve return
'Pinguim não voa. Anda no chão.' class Papagaio(Ave): def voar(self): return 'Papagaio voa alto!'
aves = [Pinguim(), Papagaio()] for ave in aves: print(ave.voar()) # mesmo método, comportamentos
diferentes # ===== # 4. ABSTRAÇÃO –
interface simples, complexidade escondida #
===== from abc import ABC, abstractmethod
class Imovel(ABC): # não pode ser instanciada diretamente @abstractmethod def
calcular_valor(self): pass @abstractmethod def get_endereco(self): pass def descricao(self):
return f'Imóvel em {self.get_endereco()}, valor R$ {self.calcular_valor():.2f}' class
Casa(Imovel): def __init__(self, endereco, metragem, preco_m2): self.endereco = endereco
self.metragem = metragem self.preco_m2 = preco_m2 def calcular_valor(self): return self.metragem *
self.preco_m2 def get_endereco(self): return self.endereco casa = Casa('Curitiba - Rebouças', 120,
8500) print(casa.descricao()) # Imóvel em Curitiba - Rebouças, valor R$ 1.020.000,00
```

Os 4 pilares são: **Encapsulamento**, **Herança**, **Polimorfismo** e **Abstração**. Python suporta todos, mas com uma abordagem mais flexível (não tem modificadores de acesso estritos como Java).

■ O que é @property? Quando usar getters e setters em Python?

```

class Produto: def __init__(self, nome, preco): self.nome = nome self.__preco = preco # privado
@property def preco(self): """Getter – acessa como atributo: produto.preco""" return self.__preco
@preco.setter def preco(self, valor): """Setter – valida antes de atribuir: produto.preco = 100"""
if valor < 0: raise ValueError('Preço não pode ser negativo') self.__preco = valor @preco.deleter
def preco(self): """Deleter – ação ao deletar: del produto.preco""" print(f'Deletando preço de
{self.nome}') del self.__preco p = Produto('Notebook', 3500) print(p.preco) # 3500 – getter
p.preco = 3200 # setter (valida) print(p.preco) # 3200 # p.preco = -100 # ValueError del p.preco #
Deleter chamado # Uso prático: cache de propriedades computadas class Circulo: def __init__(self,
raio): self._raio = raio @property def raio(self): return self._raio @property def area(self):
"""Computada – recalcula sempre, mas acessada como atributo.""" import math return math.pi *
self._raio ** 2 @property def circunferencia(self): import math return 2 * math.pi * self._raio c
= Circulo(5) print(c.area) # 78.54 – acesso como atributo print(c.circunferencia) # 31.42

```

@property permite usar métodos como se fossem atributos, criando getters, setters e deleters com sintaxe limpa. Em vez de conta.get_saldo(), usa-se conta.saldo. Permite validação transparente sem mudar a interface da classe.

1.6 Princípios SOLID

■ Explique cada princípio SOLID com exemplos em Python.

```

# ===== # S – Single Responsibility
Principle (SRP) # "Uma classe deve ter uma única razão para mudar" #
===== # ■ ERRADO – classe faz muitas
coisas class Pedido: def __init__(self, cliente, itens): self.cliente = cliente self.itens = itens
def calcular_total(self): ... def gerar_pdf(self): ... # responsibilities! def enviar_email(self):
... def salvar_no_banco(self): ... # ■ CERTO –■■■■■ class Pedido: def __init__(self, cliente,
itens): self.cliente = cliente self.itens = itens def calcular_total(self): ... class PedidoPDF:
def gerar(pedido): ... class PedidoEmail: def enviar(pedido): ... class PedidoRepository: def
salvar(pedido): ... # ===== # O –
Open/Closed Principle (OCP) # "Entidades devem estar abertas para extensão, fechadas para
modificação" # ===== # ■ ERRADO – para
novo formato, precisa modificar a classe class Relatorio: def gerar(self, formato): if formato ==
'pdf': return self.gerar_pdf() elif formato == 'csv': return self.gerar_csv() # se■■■ nouveaux
formato, precisa add novo if/elif # ■ CERTO – extensível sem modificar código existente from abc
import ABC, abstractmethod class FormatadorRelatorio(ABC): @abstractmethod def formatar(self,
dados): pass class FormatadorPDF(FormatadorRelatorio): def formatar(self, dados): return
f'{dados}' class FormatadorCSV(FormatadorRelatorio): def formatar(self, dados): return
dados.replace(',', ' ;') class Relatorio: def __init__(self, formatador: FormatadorRelatorio):
self.formatador = formatador def gerar(self, dados): return self.formatador.formatar(dados) # Para
adicionar JSON, só criar nova classe, não modifica Relatorio class
FormatadorJSON(FormatadorRelatorio): def formatar(self, dados): return f'{{"dados": "{dados}"}}' #
===== # L – Liskov Substitution Principle
(LSP) # "Objetos de uma subclasse devem poder substituir objetos da superclasse" #
===== class Ave: def voar(self): return
'voando' class Pinguim(Ave): def voar(self): # Violação! Pinguim não voa raise
NotImplementedError('Pinguim não voa') # ■ CERTO – definir interface mais granular class Ave: pass
class AveVoadora(Ave): def voar(self): return 'voando' class AveNaoVoadora(Ave): def andar(self):
return 'andando' # ===== # I – Interface
Segregation Principle (ISP) # "Clientes não devem ser forçados a depender de métodos que não usam"
# ===== # ■ ERRADO – interface gorda class
ITrabalhador: def trabalhar(self): pass def comer(self): pass def dormir(self): pass class
Robo(ITrabalhador): def trabalhar(self): pass def comer(self): raise NotImplementedError() # não
precisa def dormir(self): raise NotImplementedError() # não precisa # ■ CERTO – interfaces
pequenas e específicas class ITrabalhavel: def trabalhar(self): pass class IComestivel: def
comer(self): pass class Funcionario(ITrabalhavel, IComestivel): def trabalhar(self): pass def
comer(self): pass class Robo(ITrabalhavel): def trabalhar(self): pass #

```



```

===== # D – Dependency Inversion Principle
(DIP) # "Módulos de alto nível não devem depender de módulos de baixo nível. # Ambos devem
depende de abstrações." # ===== # ■
ERRADO – Pedido depende de MySQLConnection (concreto) class Pedido: def __init__(self): self.db =
MySQLConnection() # acoplamento forte # ■ CERTO – Pedido depende de abstração (interface) class
RepositorioPedido(ABC): @abstractmethod def salvar(self, pedido): pass @abstractmethod def
buscar(self, id): pass class Pedido: def __init__(self, repo: RepositorioPedido): # injeta via
construtor self.repo = repo # Qualquer implementação: MySQL, PostgreSQL, Mock, etc class
MySQLRepositorio(RepositorioPedido): def salvar(self, p): ... def buscar(self, id): ... class
PostgresRepositorio(RepositorioPedido): def salvar(self, p): ... def buscar(self, id): ... # Teste
com mock – inverte controle class MockRepositorio(RepositorioPedido): dados = {} def salvar(self,
p): self.dados[p.id] = p def buscar(self, id): return self.dados.get(id)

```

SOLID são 5 princípios de design orientado a objetos que tornam código mais legível, flexível e fácil de manter.

Na prática em Python, nem todos são naturais (como em linguagens tipadas), mas entender os conceitos ajuda muito em arquitetura.

1.7 Clean Code e Boas Práticas

■ Quais são os princípios de Clean Code que você aplica no dia a dia?

```

# ===== # NOMES SIGNIFICATIVOS #
===== # ■ Nomes ruins d = 30 # o quê? x = y
- 5 # confuso def fn(l): return [x**2 for x in l if x > 0] # o quê é x, l? # ■ Nomes claros
idade_cliente = 30 desconto = 5 def calcular_quadrados_positivos(lista_numeros): return [numero **
2 for numero in lista_numeros if numero > 0] # Nomes com base no domínio class AccountService:
pass # ■ class Act: pass # ■ def wait_for_response(): pass # ■ def loop(): pass # ■ #
===== # FUNÇÕES PEQUENAS #
===== # ■ Função grande – faz muitas
coisas def processar_pedido(pedido): # valida, formata, salva, envia email, atualiza inventário...
# 200 linhas de código # ■ Funções pequenas com nomes descritivos def
validar_itens_pedido(pedido): pass def calcular_desconto(pedido, cliente): pass def
salvar_pedido(pedido): pass def notificar_cliente(pedido): pass def atualizar_inventario(pedido):
pass def processar_pedido(pedido): validar_itens_pedido(pedido) desconto =
calcular_desconto(pedido, cliente) salvar_pedido(pedido) notificar_cliente(pedido)
atualizar_inventario(pedido) # ===== #
EARLY RETURN – evita else profundamente aninhado #
===== # ■ else profundamente aninhado def
processar(envio): if envio: if envio.ativo: if envio.tipo == 'email': enviar_email(envio) else:
pass # outro tipo else: pass # inativo else: raise ValueError('Envio inválido') # ■ Early return
(guard clauses) def processar(envio): if not envio: raise ValueError('Envio inválido') if not
envio.ativo: return # early exit if envio.tipo == 'email': enviar_email(envio) elif envio.tipo ==
'sms': enviar_sms(envio) # ===== #
DOCSTRINGS # ===== def
calcular_desconto(valor_original, percentual): """ Calcula o valor do desconto aplicado a um
preço. Args: valor_original (float): preço antes do desconto percentual (float): percentual de
desconto (0-100) Returns: float: valor do desconto em reais Raises: ValueError: se percentual fora
do range 0-100 Example: >>> calcular_desconto(100, 10) 10.0 """ if not 0 <= percentual <= 100:
raise ValueError('Percentual deve estar entre 0 e 100') return valor_original * (percentual / 100)
# ===== # CONSTANTES vs MAGIC NUMBERS #
===== # ■ if usuario.idade > 18 and
usuario.idade < 65: # 18 e 65 são números mágicos renderizar_conteudo() # ■ IDADE_ADULTO = 18
IDADE_MAXIMA_TRABALHADOR = 65 if IDADE_ADULTO <= usuario.idade <= IDADE_MAXIMA_TRABALHADOR:
renderizar_conteudo()

```

Clean Code é escrever código que seja: legível (outros entendem), testável, simples e reutilizável. Os princípios principais são: nomes significativos, funções pequenas, responsabilidades únicas, ■■■■■■■■■■ de

erros, e ausência de duplicação.

■ O que é DRY (Don't Repeat Yourself)? Mostre como refatorar código duplicado.

```
# ■ CÓDIGO DUPLICADO – o mesmo cálculo em 3 lugares
def calcular_preco_iphone():
    custo = 3000
    margem = 1.3
    return custo * margem
def calcular_preco_samsung():
    custo = 2800
    margem = 1.3
    return custo * margem
def calcular_preco_xiaomi():
    custo = 2000
    margem = 1.3
    return custo * margem
# ■ REFATORADO – lógica centralizada
MARGEM_LUCRO = 1.3
def calcular_preco_produto(custo, nome_produto):
    if custo <= 0:
        raise ValueError(f'Custo do {nome_produto} deve ser positivo')
    return custo * MARGEM_LUCRO
precos = {
    'iPhone': calcular_preco_produto(3000, 'iPhone'),
    'Samsung': calcular_preco_produto(2800, 'Samsung'),
    'Xiaomi': calcular_preco_produto(2000, 'Xiaomi'),
}
# ===== # TEMPLATE METHOD – evitar repetição com herança # =====
class PipelineProcessamento:
    """Classe base com o template do processo."""
    def executar(self, dados):
        dados = self.validar(dados)
        dados = self.transformar(dados)
        dados = self.salvar(dados)
        return dados
    def validar(self, dados):
        if not dados:
            raise ValueError('Dados vazios')
        return dados
    @abstractmethod
    def transformar(self, dados):
        pass
    @abstractmethod
    def salvar(self, dados):
        pass
class PipelineVendas(PipelineProcessamento):
    def transformar(self, dados):
        return [d for d in dados if d['valor'] > 0]
    def salvar(self, dados):
        # lógica específica para vendas
        return dados
class PipelineClientes(PipelineProcessamento):
    def transformar(self, dados):
        return [d for d in dados if d['ativo']]
    def salvar(self, dados):
        # lógica específica para clientes
        return dados
```

DRY significa evitar repetição de lógica. Cada conceito deve ter uma única representação no código. Código duplicado gera inconsistência na manutenção.

■ SEÇÃO 2 — ESTRUTURAS DE DADOS

Estruturas de dados são a base da ciência da computação. Em entrevistas técnicas, é comum receber problemas de lógica que exigem escolher a estrutura certa. Para júnior, geralmente cobram compreensão dos conceitos e capacidade de implementar as principais operações — não necessariamente otimizar algoritmos complexos.

■ Explique as complexidades (Big O) das principais operações em listas, dicionários, sets e árvores.

COMPLEXIDADES DAS PRINCIPAIS OPERAÇÕES:

[illegible]

Big O descreve como o tempo/espaco de um algoritmo cresce com o tamanho da entrada. Dominar isso e essencial para escolher a estrutura certa e resolver problemas de performance.

■ O que é uma pilha (stack) e uma fila (queue)? Implemente ambas com Python puro.

```
# STACK (Pilha) – LIFO
class Stack:
    def __init__(self):
        self.items = []
    def push(self, item):
        self.items.append(item) # O(1)
    def pop(self):
        if self.is_empty():
            raise IndexError('Pop from empty stack')
        return self.items.pop() # O(1)
    def peek(self):
        if self.is_empty():
            raise IndexError('Peek from empty stack')
        return self.items[-1]
    def is_empty(self):
        return len(self.items) == 0
    def __len__(self):
        return len(self.items)
    def __repr__(self):
        return f'Stack({self.items})' # Uso:
# validação de parênteses balanceados
def validar_parenteses(expr):
    stack = Stack()
    opening = '([{'
    closing = ')]}'
    pairs = {'(': ')', '{': '}', '[': ']'}
    for char in expr:
        if char in opening:
            stack.push(char)
        elif char in closing:
            if stack.is_empty():
                return False
            top = stack.pop()
            if pairs[top] != char:
                return False
    return stack.is_empty()
print(validar_parenteses('{{[]}}')) # True
print(validar_parenteses('{{}}')) # False #

===== # QUEUE (Fila) – FIFO #
=====
from collections import deque
class Queue:
    def __init__(self):
        self.items = deque() # deque é mais eficiente que list para pop(0)
    def enqueue(self, item):
        self.items.append(item) # O(1)
    def dequeue(self):
        if self.is_empty():
            raise IndexError('Dequeue from empty queue')
        return self.items.popleft() # O(1)
    def peek(self):
        if self.is_empty():
            raise IndexError('Peek from empty queue')
        return self.items[0]
    def is_empty(self):
        return len(self.items) == 0
    def __len__(self):
        return len(self.items) # Uso:
# simulação de fila de banco
fila = Queue()
fila.enqueue('Ana')
fila.enqueue('Bruno')
```

```

fila.enqueue('Carlos') print(fila.dequeue()) # 'Ana' – primeiro da fila print(fila.dequeue()) #
'Bruno' print(len(fila)) # 1 # ===== #
DEQUE – fila de dupla ponta # ===== from
collections import deque d = deque(maxlen=5) # tamanho máximo – desloca automaticamente
d.append(1) d.append(2) d.append(3) d.appendleft(0) # insere no início print(d) # deque([0, 1, 2,
3], maxlen=5) d.pop() # remove do final d.popleft() # remove do início print(d) # deque([1, 2],
maxlen=5)

```

Stack (pilha): LIFO — Last In, First Out. O último elemento adicionado é o primeiro a sair. Usa push/pop.

Exemplo: undo/redo, chamada de funções. **Queue (fila):** FIFO — First In, First Out. O primeiro elemento adicionado é o primeiro a sair. Usa enqueue/dequeue. Exemplo: fila de impressão, tarefas assíncronas.

■ O que é uma árvore binária? Implemente inserção, busca e travessia.

```

class No: def __init__(self, valor): self.valor = valor self.esquerda = None self.direita = None
class ArvoreBinaria: def __init__(self): self.raiz = None def inserir(self, valor): """Inserção em
BST – mantém ordem.""" if not self.raiz: self.raiz = No(valor) else:
self._inserir_recursivo(self.raiz, valor) def _inserir_recursivo(self, no, valor): if valor <
no.valor: if no.esquerda is None: no.esquerda = No(valor) else:
self._inserir_recursivo(no.esquerda, valor) else: if no.direita is None: no.direita = No(valor)
else: self._inserir_recursivo(no.direita, valor) def buscar(self, valor): """Busca em BST – O(log
n) se balanceada.""" return self._buscar_recursivo(self.raiz, valor) def _buscar_recursivo(self,
no, valor): if no is None: return False if valor == no.valor: return True elif valor < no.valor:
return self._buscar_recursivo(no.esquerda, valor) else: return self._buscar_recursivo(no.direita,
valor) def em_ordem(self): """In-order traversal – nós em ordem crescente.""" resultado = []
self._em_ordem_recursivo(self.raiz, resultado) return resultado def _em_ordem_recursivo(self, no,
resultado): if no: self._em_ordem_recursivo(no.esquerda, resultado) resultado.append(no.valor)
self._em_ordem_recursivo(no.direita, resultado) def pre_ordem(self): """Pre-order – útil para
copiar árvore.""" resultado = [] def traverse(no): if no: resultado.append(no.valor)
traverse(no.esquerda) traverse(no.direita) traverse(self.raiz) return resultado def
pos_ordem(self): """Post-order – útil para deletar árvore.""" resultado = [] def traverse(no): if
no: traverse(no.esquerda) traverse(no.direita) resultado.append(no.valor) traverse(self.raiz)
return resultado def altura(self, no): """Calcula altura da árvore.""" if no is None: return -1 #
-1 para árvore vazia, 0 para nó sem filhos return 1 + max(self.altura(no.esquerda),
self.altura(no.direita)) # Uso arvore = ArvoreBinaria() for val in [15, 10, 20, 8, 12, 17, 25]:
arvore.inserir(val) print(arvore.em_ordem()) # [8, 10, 12, 15, 17, 20, 25] – ordenado!
print(arvore.buscar(12)) # True print(arvore.buscar(99)) # False print(arvore.pre_ordem()) # [15,
10, 8, 12, 20, 17, 25] print(arvore.altura(arvore.raiz)) # 2

```

Árvore binária é uma estrutura hierárquica onde cada nó tem no máximo dois filhos (esquerda e direita). É a base para árvores de busca (BST), heaps, e muitas estruturas usadas em bancos de dados e sistemas de arquivos.

■ O que é tabela hash (dict)? Como funciona o hashing? Quais são as estratégias para resolver colisões?

```

# Como funciona dict internamente em Python: # 1. Calcula hash da chave: hash('nome') # 2. Usa
hash para determinar índice: hash % tamanho_tabela # 3. Armazena (hash, key, value) no slot #
Demonstração conceitual: class TabelaHash: def __init__(self, tamanho=16): self.tamanho = tamanho
self.tabela = [[] for _ in range(tamanho)] # lista de listas (encadeamento) def _hash(self,
chave): return hash(chave) % self.tamanho def put(self, chave, valor): indice = self._hash(chave)
lista = self.tabela[indice] for i, (k, v) in enumerate(lista): if k == chave: lista[i] = (k,
valor) # atualiza return lista.append((chave, valor)) # insere def get(self, chave): indice =
self._hash(chave) lista = self.tabela[indice] for k, v in lista: if k == chave: return v raise
KeyError(chave) def __contains__(self, chave): indice = self._hash(chave) lista =
self.tabela[indice] return any(k == chave for k, v in lista) # Uso ht = TabelaHash()
ht.put('nome', 'Ana') ht.put('idade', 28) ht.put('cidade', 'Curitiba') print(ht.get('nome')) #

```

```
'Ana' print(ht.get('idade')) # 28 print('cidade' in ht) # True # RESOLVENDO COLISÕES: # 1.  
Encadeamento (separate chaining) – Python usa isso # Cada slot é uma lista encadeada # Inserção  
O(1), busca O(n/k) onde k = tamanho da tabela # 2. Open Addressing – linear probing # Se slot  
ocupado, procura próximo slot vazio # hash(key) → ocupado? → +1 → +2 → ... # search: hash(key)  
→ verifica slot → próximo slot se vazio # PERFORMANCE: # Fator de carga (load factor) = n/m  
(elementos / slots) # Python redimensiona automaticamente quando load factor > 2/3 # Para garantir  
O(1), load factor deve ser baixo # Por isso dict é rápido: crescimento exponencial  
(16→32→64→...) import sys d = {} for i in range(10000): d[f'key_{i}'] = i print(f'Memory:  
{sys.getsizeof(d) / 1024:.1f} KB') print(f'Hash table size: {len(d)} entries')
```

Tabela hash é uma estrutura que mapeia chaves a valores usando uma função hash. Permite busca, inserção e remoção em O(1) em média. A função hash converte a chave em um índice do array. Colisões (quando duas chaves geram o mesmo índice) são resolvidas por encadeamento (linked list) ou open addressing (linear probing, quadratic probing).

■ SEÇÃO 3 — SQL

■ O que é JOIN? Quais os tipos e quando usar cada um? Exemplos práticos.

```
-- ESTRUTURA BASE -- Tabela: pedidos (id, cliente_id, valor, data) -- Tabela: clientes (id, nome, cidade)
-- INNER JOIN: só linhas com correspondência em ambas -- Retorna clientes que têm pedidos E pedidos que têm clientes
SELECT c.nome, p.valor, p.data FROM clientes c INNER JOIN pedidos p ON c.id = p.cliente_id;
-- Se cliente não tem pedido, não aparece -- Se pedido tem cliente_id inexistente, não aparece
-- LEFT JOIN: todas as linhas da tabela esquerda + matches da direita -- Todos os clientes aparecem, mesmo sem pedido (valor = NULL)
SELECT c.nome, p.valor FROM clientes c LEFT JOIN pedidos p ON c.id = p.cliente_id;
-- Resultado: cliente 'Ana' sem pedido → (Ana, NULL)
-- RIGHT JOIN: todas as linhas da tabela direita + matches da esquerda
SELECT c.nome, p.valor FROM clientes c RIGHT JOIN pedidos p ON c.id = p.cliente_id;
-- Resultado: pedidos sem cliente correspondentes → (NULL, valor)
-- FULL OUTER JOIN: tudo de ambas (■ + ■, NULL onde não há match)
SELECT c.nome, p.valor FROM clientes c FULL OUTER JOIN pedidos p ON c.id = p.cliente_id;
-- Reunião completa, sem perder nada
-- LEFT JOIN com filtro NULL – clientes SEM pedidos
SELECT c.nome FROM clientes c LEFT JOIN pedidos p ON c.id = p.cliente_id WHERE p.id IS NULL;
-- pedidos inexistentes
-- SELF JOIN – hierarquia de funcionários
SELECT e.nome AS funcionario, m.nome AS gerente FROM funcionarios e LEFT JOIN funcionarios m ON e.gerente_id = m.id;
-- MULTI JOIN – 3 ou mais tabelas
SELECT c.nome, p.valor, p.status FROM clientes c INNER JOIN pedidos p ON c.id = p.cliente_id INNER JOIN itens_pedido ip ON p.id = ip.pedido_id INNER JOIN produtos pr ON ip.produto_id = pr.id WHERE c.cidade = 'Curitiba';
-- CROSS JOIN – produto cartesiano (raro, usar com cuidado)
SELECT c.nome, p.nome AS produto_todos FROM clientes c CROSS JOIN produtos p;
-- Cada cliente com TODOS os produtos (n*m linhas)
```

JOIN combina linhas de duas ou mais tabelas baseado em uma condição de **■**. Os principais tipos são: INNER, LEFT, RIGHT, FULL OUTER e CROSS. A escolha depende da necessidade de manter linhas sem correspondência.

■ O que é GROUP BY e HAVING? Qual a diferença entre WHERE e HAVING?

```
-- WHERE vs HAVING – diferença crítica -- WHERE: filtra linhas ANTES do GROUP BY -- HAVING: filtra grupos DEPOIS do GROUP BY
-- Exemplo: quero clientes com mais de 5 pedidos em 2025
SELECT cliente_id, COUNT(*) AS total_pedidos FROM pedidos WHERE YEAR(data) = 2025 -- filtro ANTES da agregação
GROUP BY cliente_id -- agrupa por cliente
HAVING COUNT(*) > 5; -- filtro DEPOIS da agregação
-- TRADUÇÃO passo a passo:
-- 1. FROM pedidos – pega tabela
-- 2. WHERE YEAR(data) = 2025 – remove linhas de outros anos
-- 3. GROUP BY cliente_id – agrupa
-- 4. HAVING COUNT(*) > 5 – remove grupos com ≤ 5 pedidos
-- 5. SELECT – mostra resultado
-- EXEMPLO PRÁTICO: revenue por categoria com filtros
SELECT categoria, COUNT(*) AS total_produtos, SUM(estoque) AS estoque_total, AVG(preco) AS preco_medio FROM produtos WHERE ativo = 1 -- só produtos ativos
GROUP BY categoria
HAVING SUM(estoque) > 100 -- só categorias com estoque significativo
ORDER BY SUM(estoque) DESC;
-- ordena por estoque total
-- AGREGADOS com CASE – contagem condicional
SELECT c.nome, COUNT(p.id) AS total_pedidos, SUM(CASE WHEN p.status = 'atrasado' THEN 1 ELSE 0 END) AS pedidos_atrasados,
AVG(CASE WHEN p.status = 'concluido' THEN p.valor ELSE NULL END) AS ticket_medio FROM clientes c LEFT JOIN pedidos p ON c.id = p.cliente_id
GROUP BY c.id, c.nome
HAVING AVG(CASE WHEN p.status = 'concluido' THEN p.valor ELSE NULL END) > 500
ORDER BY total_pedidos DESC;
-- Funções de agregação:
-- COUNT(*) – todas as linhas
-- COUNT(coluna) – não conta NULLs
-- COUNT(DISTINCT coluna) – valores únicos
-- SUM, AVG, MIN, MAX – ignoram NULLs
-- STRING_AGG – concatena valores (PostgreSQL)
```

GROUP BY agrupa linhas com valores iguais em colunas agregadas. HAVING filtra grupos após a agregação (equivalente ao WHERE para grupos). WHERE filtra linhas individuais ANTES da agregação.

■ Como otimizar uma query SQL lenta? Passo a passo completo.

```
-- ===== -- PASSO 1: IDENTIFICAR QUERY
LENTA -- ===== -- PostgreSQL SELECT query,
calls, total_exec_time, mean_exec_time, stddev_exec_time FROM pg_stat_statements ORDER BY
total_exec_time DESC LIMIT 10; -- MySQL SHOW PROCESSLIST; -- queries em execução SELECT * FROM
performance_schema.events_statements_summary_by_digest ORDER BY SUM_TIMER_WAIT DESC LIMIT 10; --
===== -- PASSO 2: EXPLAIN ANALYZE – ver o
plano de execução -- ===== EXPLAIN
(ANALYZE, BUFFERS, FORMAT TEXT) SELECT c.nome, COUNT(p.id) AS total FROM clientes c LEFT JOIN
pedidos p ON c.id = p.cliente_id WHERE c.cidade = 'Curitiba' GROUP BY c.id, c.nome; -- No plano,
procurar: -- Seq Scan – tabela inteira, pode precisar índice -- Nested Loop – pode ser lento para
grandes volumes -- Hash Join – bom para grandes volumes -- Sort – se mostrando em grandes tabelas,
criar índice -- ===== -- PASSO 3: CRIAR
ÍNDICES -- ===== -- Índice simples CREATE
INDEX idx_clientes_cidade ON clientes(cidade); -- Índice composto (para WHERE com múltiplas
colunas) CREATE INDEX idx_pedidos_cliente_status ON pedidos(cliente_id, status); -- Índice para
LIKE com prefixo CREATE INDEX idx_produtos_nome ON produtos(nome varchar_pattern_ops); --
Verificar se índice é usado EXPLAIN SELECT * FROM pedidos WHERE cliente_id = 123; --
===== -- PASSO 4: EVITAR PROBLEMAS COMUNS
-- ===== -- ■ SELECT * – carrega colunas
desnecessárias SELECT * FROM pedidos; -- não faça isso -- ■ Selecionar só o necessário SELECT id,
data, valor FROM pedidos WHERE cliente_id = 123; -- ■ Função na coluna do WHERE – impede uso de
índice SELECT * FROM pedidos WHERE YEAR(data) = 2025; SELECT * FROM pedidos WHERE MONTH(data) = 6;
-- ■ Range de datas – permite índice SELECT * FROM pedidos WHERE data >= '2025-01-01' AND data <
'2026-01-01'; -- ■ LIKE com wildcard no início SELECT * FROM produtos WHERE nome LIKE '%celular%';
-- ■ LIKE com wildcard só no fim SELECT * FROM produtos WHERE nome LIKE 'celular%'; -- Usa índice
se existir -- ■ Subquery no WHERE – pode ser lento SELECT * FROM clientes WHERE id IN (SELECT
cliente_id FROM pedidos WHERE valor > 1000); -- ■ JOIN vs subquery – geralmente mais eficiente
SELECT c.* FROM clientes c INNER JOIN pedidos p ON c.id = p.cliente_id WHERE p.valor > 1000; -- ■
Múltiplas subqueries correlacionadas -- (subquery que depende da query externa) --
===== -- PASSO 5: VERIFICAR RESULTADO --
===== EXPLAIN ANALYZE SELECT ... →
comparar tempos antes/depois -- Índice não usado? Verificar: -- 1. Estatísticas atualizadas?
ANALYZE; -- 2. Coluna tem alta cardinalidade? -- 3. Query usa a mesma coluna do índice? -- DROP
índices não usados DROP INDEX idx_ nao_usado;
```

Otimização de queries é uma habilidade crítica. O processo é: identificar a query lenta → usar EXPLAIN → analisar plano → aplicar otimizações → verificar resultado.

■ O que é normalização? Explique 1NF, 2NF e 3NF com exemplos.

```
-- ===== -- 1NF – First Normal Form (Forma
Normal 1) -- Regra: cada célula contém apenas valor atômico (sem listas ou tabelas dentro) --
===== -- ■ VIOLA 1NF – coluna com
múltiplos valores CREATE TABLE usuarios ( id INT, nome VARCHAR(100), telefones VARCHAR(200) --
'99999-1111, 88888-2222' – múltiplos valores! ); -- ■ CORRIGIDO – para tabela separada CREATE
TABLE telefones ( usuario_id INT, telefone VARCHAR(20), PRIMARY KEY (usuario_id, telefone) ); --
===== -- 2NF – Second Normal Form -- Regra:
Está em 1NF + não há dependência parcial de chave composta --
===== -- ■ VIOLA 2NF – dependência parcial
de chave composta CREATE TABLE notas ( aluno_id INT, disciplina_id INT, nome_aluno VARCHAR(100),
-- depende só de aluno_id, não de disciplina! nota DECIMAL(5,2), PRIMARY KEY (aluno_id,
disciplina_id) ); -- Problema: nome_aluno se repete para cada disciplina do mesmo aluno -- ■
CORRIGIDO – tabela separada para alunos CREATE TABLE alunos ( id INT PRIMARY KEY, nome
VARCHAR(100) ); CREATE TABLE notas ( aluno_id INT REFERENCES alunos(id), disciplina_id INT, nota
DECIMAL(5,2), PRIMARY KEY (aluno_id, disciplina_id) ); --
===== -- 3NF – Third Normal Form -- Regra:
Está em 2NF + não há dependência transitiva --
```

```

===== -- ■ VIOLA 3NF – coluna depende de
outra coluna que não é chave CREATE TABLE pedidos ( id INT PRIMARY KEY, cliente_id INT,
cidade_cliente VARCHAR(100), -- depende de cliente_id, não de pedido! -- cidade_cliente depende de
cliente_id, que depende de id ); -- Problema: cidade se repete em todos pedidos do mesmo cliente
-- ■ CORRIGIDO – cidade está na tabela de clientes CREATE TABLE clientes ( id INT PRIMARY KEY,
cidade VARCHAR(100) -- cidade pertence aqui ); CREATE TABLE pedidos ( id INT PRIMARY KEY,
cliente_id INT REFERENCES clientes(id) ); --
===== -- QUANDO DESNORMALIZAR --
===== -- Desnormalização = adicionar
redundância intencional por performance -- Exemplo: relatório mensal de vendas -- Em vez de JOIN
em 3 tabelas para cada linha do relatório, -- criar tabela agregada pré-computada CREATE TABLE
relatorio_vendas_mensal ( mes DATE, categoria VARCHAR(50), total_vendas DECIMAL(15,2),
quantidade_pedidos INT, updated_at TIMESTAMP ); -- Atualiza via job noturno (ou trigger) --
Relatório que antes demorava 30s agora retorna em 100ms

```

Normalização é o processo de organizar dados em tabelas para minimizar redundância e evitar anomalias (insert/update/delete). Cada forma normal (NF) tem regras específicas que eliminam certos tipos de problemas.

■ O que é transação? Explique ACID e os comandos COMMIT e ROLLBACK.

```

-- TRANSAÇÃO BÁSICA BEGIN TRANSACTION; UPDATE contas SET saldo = saldo - 100 WHERE id = 1; UPDATE
contas SET saldo = saldo + 100 WHERE id = 2; -- Se ambos succeeds COMMIT; -- Se qualquer falha
acontecer no meio -- ROLLBACK; – reverte tudo --
===== -- ACID – propriedades garantidas
por transações -- ===== -- ATOMICITY
(Atomicidade) -- "Tudo ou nada" – transação não pode ser parcialmente commitada BEGIN; UPDATE
estoque SET qtd = qtd - 1 WHERE produto_id = 5; UPDATE vendas SET total = total + 100 WHERE id =
1; -- Se uma falhar, todas são revertidas COMMIT; -- CONSISTENCY (Consistência) -- O banco vai de
um estado válido para outro -- Constraints, chaves, triggers são respeitadas -- ISOLATION
(Isolamento) -- Transações concorrentes não se interferem -- Níveis: READ UNCOMMITTED < READ
COMMITTED < REPEATABLE READ < SERIALIZABLE SET TRANSACTION ISOLATION LEVEL SERIALIZABLE; BEGIN;
--Esta transação vê dados consistentes até commit COMMIT; -- DURABILITY (Durabilidade) --
Commitado = salvo mesmo em crash -- Banco usa write-ahead log (WAL), replication, backup --
===== -- EXEMPLO PRÁTICO: transferência
bancária -- ===== BEGIN; -- 1. Debitar da
conta origem UPDATE contas SET saldo = saldo - 500 WHERE id = 101 AND saldo >= 500; -- Se saldo
insuficiente, a condição falhar e nenhuma linha for afetada -- → verificar rows_affected e fazer
ROLLBACK -- 2. Creditar na conta destino UPDATE contas SET saldo = saldo + 500 WHERE id = 202; --
3. Registrar no histórico INSERT INTO transferencias (origem, destino, valor, data) VALUES (101,
202, 500, NOW()); -- 4. Confirmar ou reverter -- Se nenhuma falha: COMMIT -- Se qualquer problema:
ROLLBACK COMMIT; -- ===== -- SAVEPOINT –
rollback parcial -- ===== BEGIN; INSERT
INTO pedidos (cliente_id, valor) VALUES (1, 100); SAVEPOINT sp1; INSERT INTO itens_pedido
(pedido_id, produto_id, qtd) VALUES (1, 10, 2); -- Se algo falhar aqui... ROLLBACK TO SAVEPOINT
sp1; -- reverte só o segundo INSERT -- O primeiro INSERT permanece COMMIT; -- confirma só o
primeiro INSERT

```

Transação é um conjunto de operações SQL que devem ser executadas atomicamente – ou todas completam, ou nenhuma altera o banco. ACID garante consistência e confiabilidade.

■ SEÇÃO 4 — RAG e LLMs

■ O que é RAG? Descreva o fluxo completo do zero à resposta.

FLUXO COMPLETO RAG – DO DOCUMENTO À RESPOSTA: ETAPA 1 – INGESTÃO (■■■)

Documentos

original (PDF, HTML, texto) ■ ■ ↓ ■ ■ Chunking – dividir em pedaços menores (500-1000 tokens) ■ ■ - Fixed-size: blocos de 500 tokens, overlap de 50-100 ■ ■ - Semantic: por parágrafos ou seções (mais preciso) ■ ■ - Recursive: tenta manter contexto semântico ■ ■ ↓ ■ ■ Embedding – converter cada chunk em vetor numérico ■ ■ Modelo: text-embedding-3-small (1536 dims) ou similar ■ ■ OpenAI: embeddings = client.embeddings.create(...) ■ ■ ↓ ■ ■ Indexação – armazenar vetores + chunks no banco vetorial ■ ■ Qdrant, Pinecone, Weaviate, pgvector, Chroma, Milvus ■

CONSULTA (Retrieval) ETAPA 2 -

GERAÇÃO (Generation) ETAPA 3 –

```

# CÓDIGO
COMPLETO – RAG SIMPLES COM LANGCHAIN + OPENAI
from langchain_openai import OpenAIEmbeddings,
ChatOpenAI from langchain_community.vectorstores import Chroma from langchain.text_splitter import
RecursiveCharacterTextSplitter from langchain.chains import RetrievalQA # 1. Carregar documento
from langchain_community.document_loaders import PyPDFLoader loader =
PyPDFLoader('bari_home_equity.pdf') docs = loader.load() # 2. Chunking splitter =
RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50) chunks =
splitter.split_documents(docs) # 3. Embedding + indexing embeddings =
OpenAIEmbeddings(model='text-embedding-3-small') vectorstore = Chroma.from_documents(chunks,
embeddings, persist_directory='./db') # 4. Configurar LLM llm = ChatOpenAI(model='gpt-4o-mini',
temperature=0.3) # 5. Cadeia RAG chain = RetrievalQA.from_chain_type( llm=llm,
retriever=vectorstore.as_retriever(search_kwargs={'k': 5}), chain_type='stuff' # coloca todos
chunks no prompt ) # 6. Perguntar resposta = chain.invoke({'query': 'como funciona home equity no
Bari?'}) print(resposta['result'])

```

RAG (Retrieval-Augmented Generation) combina busca semântica com geração de texto por LLMs. Permite que modelos respondam com base em documentos específicos, com citing de fontes, reduzindo hallucination e mantendo dados atualizados.

■ O que é embedding? Como escolher o modelo de embedding ideal?

```
# EMBEDDING – representação vetorial de texto # Exemplo: frases e seus vetores frase1 =
"Empréstimo com garantia de imóvel" frase2 = "Home equity para sua casa" frase3 = "Câmbio de
dólar" # Todos geram vetores de 1536 dimensões (text-embedding-3-small) vec1 =
modelo.encode(frase1) # [0.12, -0.34, 0.89, ...] vec2 = modelo.encode(frase2) # [0.14, -0.31,
0.92, ...] – próximo de vec1! vec3 = modelo.encode(frase3) # [-0.78, 0.45, 0.12, ...] – distante #
```

```

Similaridade por coseno from sklearn.metrics.pairwise import cosine_similarity sim_12 =
cosine_similarity([vec1], [vec2])[0][0] # ~0.95 (muito próximo) sim_13 = cosine_similarity([vec1],
[vec3])[0][0] # ~0.12 (distante) # MODELOS DE EMBEDDING – comparação # OpenAI (pago, alta
qualidade) # - text-embedding-3-small (1536 dims, $0.02/1M tokens) – melhor custo # -
text-embedding-3-large (3072 dims, $0.13/1M tokens) – melhor qualidade # - text-embedding-ada-002
(1536 dims, $0.10/1M tokens) – deprecated # Open Source (grátis) # -
sentence-transformers/all-MiniLM-L6-v2 (384 dims) – rápido, bom # -
sentence-transformers/all-mpnet-base-v2 (768 dims) – melhor qualidade open # - bge-large-zh-v1.5
(1024 dims) – excelente para português # COMO ESCOLHER: # 1. Idioma: modelos multilingual (bge,
embigrate) vs English-only # 2. Dimensão: menor = mais rápido, maior = mais preciso # 3. Custo:
OpenAI API vs rodando local (sentence-transformers) # 4. Latência: API = rede, local = GPU # Para
embedding em português brasileiro: from sentence_transformers import SentenceTransformer modelo =
SentenceTransformer('sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2') # Suporta
português, espanhol, inglês, etc. # 384 dimensões, rápido, roda em CPU # EXEMPLO DE USO chunks =
["texto1", "texto2", "texto3"] vetores = modelo.encode(chunks) # vetores.shape = (3, 384) – 3
chunks, 384 dimensões cada # ARMAZENAR NO BANCO VETORIAL # Qdrant, Chroma, pgvector aceitam
vetores de qualquer dimensão

```

Embedding é a representação numérica (vetor) de texto em um espaço multidimensional. Textos semanticamente similares ficam próximos no espaço vetorial. A escolha do modelo afeta diretamente a qualidade da busca semântica.

■ O que é chunking? Quais estratégias existem e qual escolher?

```

ESTRATÉGIAS DE CHUNKING: 1. FIXED-SIZE (mais simples) - Divide por número de caracteres ou tokens
- Rápido, determinístico, previsível - Problema: pode cortar no meio de frases/parágrafos 2.
RECURSIVE CHARACTER (recomendado para a maioria) - Tenta dividir em blocos lógicos (parágrafos,
quebras de linha) - Mantém contexto semântico junto - langchain: RecursiveCharacterTextSplitter 3.
SEMANTIC (por sentença/seção) - Usa modelo para identificar quebras naturais - Ideal para
documentos bem estruturados - Mais lento, mas melhor qualidade 4. PARENT DOCUMENT (mais avançado)
- Chunking hierárquico: chunks pequenos para busca, documento pai maior para contexto # EXEMPLO –
Python com LangChain from langchain.text_splitter import RecursiveCharacterTextSplitter #
Configuração por tipo de documento splitter = RecursiveCharacterTextSplitter( chunk_size=500, #
tokens por chunk (não caracteres!) chunk_overlap=50, # sobreposição para não perder contexto
separators=['\n\n', # prioridade: parágrafos primeiro '\n', '. ', ' ', ''], length_function=lambda
text: len(text.split()) # Função de contagem: caracteres, tokens, palavras ) # Para documentos
técnicos (manuais, contratos) splitter_tecnico = RecursiveCharacterTextSplitter( chunk_size=800,
chunk_overlap=100, separators=['\n## ', '\n# ', '\n\n', '\n', '. ', ' '] ) # Para documentos
legais (alta precisão) splitter_legal = RecursiveCharacterTextSplitter( chunk_size=300, # chunks
menores para não perder nuances chunk_overlap=30, separators=['\n\nARTIGO', '\n\n', '\n', '. '] )
# GUIDAS PRÁTICAS: # - 500 tokens: bom para QA simples # - 800-1000 tokens: bom para documentos
técnicos # - 1500+ tokens: melhor para resumos, mas ■■■ piores # - Overlap: 10-20% do chunk size
para manter contexto # COMO MEDIR QUANTOS TOKENS UM TEXTO TEM: import tiktoken enc =
tiktoken.get_encoding('cl100k_base') # GPT-4 encoding tokens = enc.encode("texto de exemplo")
print(len(tokens)) # número de tokens

```

Chunking é dividir documentos grandes em pedaços menores para embedding. A estratégia de chunking afeta diretamente a qualidade do retrieval — chunks muito pequenos perdem contexto, muito grandes diluem informação.

■ O que é LangChain? Para que serve? Quais os componentes principais?

```

LANGCHAIN – COMPONENTES PRINCIPAIS: 1. PROMPT TEMPLATES from langchain.prompts import
PromptTemplate prompt = PromptTemplate.from_template( """Você é assistente do {empresa}. Contexto:
{contexto} Pergunta: {pergunta} Responda de forma clara e cite as fontes.""" ) formatted =
prompt.format( empresa="Banco Bari", contexto="Home equity é um empréstimo com garantia de
imóvel...", pergunta="Como funciona?" ) 2. LLMs (modelos de linguagem) from langchain_openai

```

```

import ChatOpenAI llm = ChatOpenAI(model='gpt-4o-mini', temperature=0.3)
3. CHAINS (cadeias de operações) from langchain.chains import LLMChain chain = LLMChain(llm=llm, prompt=prompt)
resposta = chain.invoke({'contexto': '...', 'pergunta': '...'})
4. RETRIEVAL + RAG from langchain.chains import RetrievalQA qa_chain = RetrievalQA.from_chain_type( llm=llm,
retriever=vectorstore.as_retriever(k=5), chain_type='stuff' )
5. AGENTS (agentes autônomos) from langchain.agents import Agent, Tool from langchain.agents.agent_types import AgentType
tools = [ Tool(name='search', func=search_func, description='busca em documentação'), Tool(name='calculator', func=calc_func, description='calcula valores') ]
agent = Agent(agent_type=AgentType.CHAT_CONVERSATIONAL_ZERO_SHOT_REACT_DESCRIPTION, llm=llm, tools=tools )
resposta = agent.invoke('Qual o valor de home equity para um imóvel de R$500mil?') # → Agent decide: usa tool search → responde
6. MEMORY (memória de conversas) from langchain.memory import ConversationBufferMemory memory = ConversationBufferMemory(memory_key='historico')
chain = LLMChain(llm=llm, prompt=prompt, memory=memory) # Permite contexto de conversas anteriores
7. OUTPUT PARSERS (parsing de respostas) from langchain.output_parsers import StructuredOutputParser, ResponseSchema
schema = ResponseSchema(name='valor', description='valor do empréstimo em reais')
parser = StructuredOutputParser(response_schemas=[schema])
chain = LLMChain(llm=llm, prompt=prompt, output_parser=parser)
resultado = chain.invoke({'pergunta': '...'}) # resultado['valor'] → 'R$ 350.000,00'
# CADEIA COMPLEXA – Multi-Step from langchain.chains import SequentialChain
chain1 = LLMChain(llm=llm, prompt=prompt_resumo, output_key='resumo')
chain2 = LLMChain(llm=llm, prompt=prompt_perguntas, input_key='resumo', output_key='perguntas')
full_chain = SequentialChain(chains=[chain1, chain2], input_variables=['documento'])
resultado = full_chain({'documento': texto_longo})

```

LangChain é um framework para construir aplicações com LLMs. Fornece abstrações para: prompting, chains (cadeias de operações), agents (agentes autônomos), memory, tools, e retrieval. Permite criar pipelines complexos combinando múltiplas etapas.

■ O que é prompt engineering? Liste e explique as principais técnicas.

PRINCIPAIS TÉCNICAS DE PROMPT ENGINEERING:

- ZERO-SHOT** (sem exemplos) prompt = "Classifique: esse email é spam ou não spam?\nEmail: você ganhou R\$1 milhão!"
- FEW-SHOT** (com exemplos) prompt = ""Classifique sentimentos como POSITIVO, NEGATIVO ou NEUTRO. Exemplos: Input: 'Adorei o produto, superou expectativas!' → POSITIVO Input: 'Chegou quebrado, péssima experiência.' → NEGATIVO Input: 'O produto chegou na terça.' → NEUTRO Classifique: 'Gostei muito do atendimento'""
- CHAIN-OF-THOUGHT (CoT)** – raciocínio passo a passo prompt = ""Resolva o problema: João tem 5 maçãs, deu 3 para Maria. Pensamento: primeiro entendo... depois calculo... portanto... Resultado:"" # Pedir para 'pensar passo a passo' melhora resultados em tasks de lógica
- TREE-OF-THOUGHTS** – explora múltiplos caminhos ""Análise de 3 formas diferentes: 1. Perspectiva financeira 2. Perspectiva de risco 3. Perspectiva de compliance Para cada perspectiva, avalie prós e contras.""
- REACT** – raciocínio + ação ""Você é um assistente de análise de crédito. Para cada pergunta: 1. RACIONAL: o que preciso verificar? 2. AÇÃO: qual ferramenta usar? 3. OBSERVAÇÃO: resultado 4. FINAL: resposta""
- SYSTEM PROMPT** – instrução de comportamento ""Você é um assistente bancário do Banco Bari. - Seja formal mas acessível - Nunca invente dados – diga 'não tenho essa informação' - Cite sempre as fontes quando disponível - Max 3 linhas por resposta""
- JSON MODE / STRUCTURED OUTPUT** # OpenAI response = client.chat.completions.create(model='gpt-4o', messages=[{'role': 'user', 'content': 'Extraia dados...'}], response_format={'type': 'json_object', 'schema': {...}}) # Anthropic response = client.messages.create(model='claude-3-5-sonnet', messages=[{'role': 'user', 'content': '...'}], response_format={'type': 'json_object', 'json_schema': {...}})
- FEW-SHOT COM CHAIN-OF-THOUGHT** ""Resolva: se um trem viaja 60km/h por 2 horas, quantos km percorre? Pensamento: velocidade × tempo = 60 × 2 = 120km Resposta: 120km Resolva: um produto custa R\$200 com 10% desconto...""
- CONTEXT INJECTION** – injetar contexto relevante prompt = f""Use este contexto para responder: --- {contexto_chunk_1} --- {contexto_chunk_2} --- Pergunta: {pergunta_usuario}""
- PERSONA + CONSTRAINTS** ""Você é [PERSONA]: advogado especialista em direito do consumidor. [REGRAS]: - Sempre cite artigos da Lei 8078/90 quando aplicável - Nunca presuma acordos sem documento - Responda em até 2 parágrafos"" # CONFIGURAÇÕES IMPORTANTES: # temperature: 0.0 = determinístico, 0.7 = criativo, 1.0 = caótico # top_p: controla diversidade (0.9 = 90% da massa de probabilidade)

```
# max_tokens: limita tamanho da resposta
```

Prompt engineering é a arte de crafting prompts eficazes para LLMs. Cada técnica serve para diferentes objetivos: clearer answers, reasoning, structured output, etc.

■ ■ SEÇÃO 5 — AWS

A vaga pede conhecimento em AWS (Amazon Web Services). Para júnior, geralmente testam compreensão dos serviços principais (EC2, S3, Lambda, RDS, IAM) e capacidade de desenhar arquiteturas simples. Profundidade em soluções enterprise não é esperada, mas saber os conceitos fundamentais é mandatório.

■ Explique os principais serviços AWS usados em projetos de IA: S3, Lambda, SageMaker, Bedrock, Rekognition.

AWS – SERVIÇOS PRINCIPAIS PARA IA:

```

1. AMAZON S3 – Object
Storage - Objetos
(arquivos) em buckets (containers) - 11 nozes de durabilidade (99.999999999%) - Camadas de storage
(S3 Standard, IA, Glacier para archive) - Uso: armazenar documentos para RAG, modelos treinados,
datasets CLI: aws s3 cp documento.pdf s3://meu-bucket/docs/ aws s3 sync ./pasta/
s3://meu-bucket/docs/ Boto3: import boto3 s3 = boto3.client('s3') s3.upload_file('modelo.pt',
'bucket', 'modelo.pt')

```

2. AWS LAMBDA – Serverless Compute

- Executa código sem provisionar servidores - Paga por execução (100ms granularidade) - Limite: 15 min por execução - Uso: API de inference leve, triggers S3, integração Exemplo: função Lambda para gerar embeddings

```
import json
import boto3

def handler(event, context):
    texto = event['text']
    # chamar embeddings API
    return {'embeddings': vetor}

# Gatilho: S3 coloca arquivo → Lambda dispara
```

3. AMAZON BEDROCK –

LLMs gerenciados (RECOMENDADO para IA)

- Claude (Anthropic), Titan (AWS), Llama (Meta), Mistral - API única para múltiplos modelos - RAG nativo com Knowledge Bases - Bedrock Agents para agentes autônomos Python SDK:

```
import boto3  
bedrock = boto3.client('bedrock-runtime')  
  
body = json.dumps({  
    'prompt': pergunta,  
    'max_tokens': 500,  
    'temperature': 0.3  
)  
response = bedrock.invoke_model(  
    modelId='anthropic.claude-3-sonnet-20240229-v1:0',  
    body=body,  
    contentType='application/json'  
)  
result = json.loads(response['body'].read())
```

ML completo (para treinamento)

- Treinar, tune, deploy de modelos - SageMaker Studio (IDE visual) - Built-in algorithms (XGBoost, Linear Learner, etc.) - Endpoint para inference em produção - Custos altos – usar para treinamento, não para inference leve

Use para fine-tuning: `sagemaker = boto3.client('sagemaker')` # cria training job com dataset customizado

4. AMAZON SAGEMAKER –

5.

AWS oferece serviços específicos para workloads de IA/ML. A escolha depende de se você quer controle total (EC2) ou managed service (Bedrock/SageMaker).

■ O que é IAM? Explique Users, Groups, Roles e Policies.

IAM – COMPONENTES:

- 1. IAM USER** – identidade para pessoa/aplicação específica - Credenciais de longo prazo (access key + secret) - Cada pessoa/app deveria ter seu próprio user - NÃO usar para aplicações em produção (use Roles)
- aws iam create-user --user-name desenvolvedor**
- aws iam create-access-key --user-name desenvolvedor**
- 2. IAM GROUP** – agrupamento de users com políticas comuns - Facilita gerenciamento: grupo DevOps, grupo Analistas - Políticas anexadas ao grupo se aplicam a todos os membros
- aws iam create-group --group-name data-team**
- aws iam add-user-to-group --user-name ana --group-name data-team**
- 3. IAM ROLE** – identidade para serviços ou acesso temporário - "Confia" em entidades (EC2, Lambda, outro account) - Credenciais temporárias auto-rotativas - Uso: EC2 acessa S3, Lambda acessa DynamoDB
- # Papel de confiança (trust policy)**
- { "Version": "2012-10-17", "Statement": [{ "Effect": "Allow", "Principal": { "Service": "lambda.amazonaws.com" }, "Action": "sts:AssumeRole" }] }**
- aws iam create-role \ --role-name lambda-s3-role \ --assume-role-policy-document file://trust-policy.json**
- 4. IAM POLICY** – documento que define permissões (JSON)
- # Policy gerenciada AWS (AWS managed)**
- { "Version": "2012-10-17", "Statement": [{ "Effect": "Allow", "Action": ["s3:GetObject", "s3:PutObject"], "Resource": "arn:aws:s3:::meu-bucket/*" }] }**
- # Anexar policy a role**
- aws iam attach-role-policy \ --role-name lambda-s3-role \ --policy-arn arn:aws:iam::aws:policy/AmazonS3FullAccess**

EXEMPLO COMPLETO – Lambda com acesso a S3:

- 1. Criar role com trust policy (Lambda pode assumir)**
- 2. Anexar policy (S3 read/write)**
- 3. Lambda usa essa role ao executar**

```
aws iam create-role \ --role-name chatbot-lambda-role \ --assume-role-policy-document '{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Principal":{"Service":"lambda.amazonaws.com"},"Action":"sts:AssumeRole"}]}'
```

```
aws iam attach-role-policy \ --role-name chatbot-lambda-role \ --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
```

Lambda no console: selecionar role chatbot-lambda-role

BOT03 – assumir role temporariamente

```
import boto3
sts = boto3.client('sts')
creds = sts.assume_role( RoleArn='arn:aws:iam::123456789:role/chatbot-role',
RoleSessionName='chatbot-session' )
```

IAM (Identity and Access Management) gerencia quem pode fazer o quê na sua conta AWS. É o service mais

AWM (Identity and Access Management) gerencia quem pode fazer o que na sua conta AWS. É o service mais crítico para segurança. O princípio fundamental é **least privilege** (mínimo privilégio necessário).

■ O que é VPC? Explique subnets, NAT Gateway, Security Groups e NACLs.

[illegible]

COMPLETO – requisição de usuário Usuário → ALB (subnet pública) → EC2 app (subnet privada) ↓ RDS (subnet privada) EC2 app precisa de dados externos → rota 0.0.0.0/0 → NAT → IGW → Internet Resposta volta pelo NAT (mantém estado)

VPC (Virtual Private Cloud) é uma rede virtual isolada na AWS. Permite controle total da rede, incluindo subnets públicas/privadas, rotas, firewalls lógicos (Security Groups e NACLs).

■ O que é CloudWatch? Quais métricas monitorar em uma aplicação de IA em produção?

CLOUDWATCH – MONITORAMENTO: 1. MÉTRICAS BÁSICAS (sempre monitorar) EC2: - CPUUtilization – uso de CPU - NetworkIn/Out – tráfego de rede - StatusCheckFailed – kesehatan instância Lambda: - Invocations – número de execuções - Duration – tempo de execução - Errors – falhas - Throttles – execuções limitadas por hitting do limite S3: - NumberOfObjects – total de objetos - BucketSizeBytes – tamanho do bucket - Requests GET/PUT – número de requisições RDS: - CPUUtilization – uso do banco - DatabaseConnections – conexões ativas - WriteIOPS / ReadIOPS – operações de disco 2. CUSTOM METRICS – métricas de negócio Python boto3: import boto3 cw = boto3.client('cloudwatch') # Enviar métrica personalizada (ex: latência de embedding) cw.put_metric_data(Namespace='ChatbotBari', MetricData=[{ 'MetricName': 'EmbeddingLatency', 'Value': 0.325, # segundos 'Unit': 'Seconds', 'Dimensions': [{ 'Name': 'Stage', 'Value': 'production'}] }]) # Enviar métrica de contagem (ex: requisições RAG) cw.put_metric_data(Namespace='ChatbotBari', MetricData=[{ 'MetricName': 'RAGRequests', 'Value': 1, 'Unit': 'Count', 'Dimensions': [{ 'Name': 'Model', 'Value': 'gpt-4o-mini'}, { 'Name': 'Stage', 'Value': 'production'}] }]) 3. ALARMES – notificar quando métrica estoura threshold aws cloudwatch put-metric-alarm \ --alarm-name high-embedding-latency \ --metric-name EmbeddingLatency \ --namespace ChatbotBari \ --statistic Average \ --period 300 \ --threshold 1.0 \ --comparison-operator GreaterThanThreshold \ --evaluation-periods 2 \ --alarm-actions arn:aws:sns:...:my-topic 4. DASHBOARDS – visualização personalizada # Dashboard JSON (criar via console ou CLI) { "widgets": [{ "type": "metric", "properties": { "title": "RAG Performance", "metrics": [["ChatbotBari", "EmbeddingLatency", "*", "production"], [".", "RAGRequests", "Model", "gpt-4o-mini"]], "period": 300, "stat": "Average" } }] } 5. LOGS – CloudWatch Logs import logging import boto3 # Configurar logging para Lambda logger = boto3.client('logs') def handler(event, context): logger.put_log_events(logGroupName='/aws/lambda/chatbot-bari', logStreamName='2025/06/01/production', logEvents=[{ 'timestamp': int(time.time() * 1000), 'message': f'Requisição: {event}' }]) 6. MONITORAMENTO DE CUSTO DE LLM # Tracking de custos por modelo (manual) def calcular_custo(tokens_input, tokens_output, modelo): precos = { 'gpt-4o-mini': {'input': 0.15, 'output': 0.60}, # por 1M tokens 'claude-3-haiku': {'input': 0.25, 'output': 1.25} } p = precos[modelo] custo = (tokens_input / 1_000_000 * p['input'] + tokens_output / 1_000_000 * p['output']) return round(custo, 6) # Registrar no CloudWatch cw.put_metric_data(Namespace='LLMCosts', MetricData=[{ 'MetricName': 'EmbeddingCost', 'Value': custo, 'Unit': 'None', 'Dimensions': [{ 'Name': 'Model', 'Value': modelo}] }]) 7. CLOUDWATCH APPLICATION INSIGHTS – para aplicações .NET/Java # Detecta automaticamente anomalias e errors aws application-insights create-application \ --resource-group-name chatbot-resource-group \ --auto-config TRACKING essentials para IA em produção: - Latência de inference (end-to-end) - Taxa de erro (4xx, 5xx) - Custo por Requisição (token usage) - Quality metrics (se disponível) - Uptime/SLA

CloudWatch é o serviço de monitoring e observability da AWS. Coleta métricas, logs e eventos. Para aplicações de IA em produção, o monitoring é crítico para detectar degradação de modelos, custos inesperados e falhas.

■ ■ SEÇÃO 6 — SYSTEM DESIGN (Básico)

■ Como você desenharia uma arquitetura para um chatbot de IA com RAG?

ARQUITETURA – CHATBOT RAG DO BANCO BARI:

```

■ (App, Web, WhatsApp, etc.) ■
■ HTTPS ▼
CLOUDFRONT (CDN) ■ Cache de conteúdo estático ■
■ ▼
■ API
GATEWAY / LOAD BALANCER ■ Rate limiting, autenticação, roteamento ■
■ ▼
FASTAPI /
LAMBDA ■ (Backend: APIs de busca e inference) ■ POST /chat ← Recebe pergunta
do usuário → Embedding da pergunta (text-embedding-3-small) → Busca
no banco vetorial (Qdrant) → top_k=5 → Monta prompt com contexto →
Chama LLM (Bedrock GPT-4o-mini) → resposta GET /health ← Health check ■
■ ▼ ▼ ▼
QDRANT ■ BEDROCK ■ S3 ■ (Vector DB) ■ (LLM
API) ■ (Documentos) ■ - Embeddings ■ - Claude ■ - PDFs ■ - Semantic ■
- GPT-4o ■ - Manuais ■ search ■ - Titan ■ - Políticas ■
COMPONENTES: 1. FRONTEND (Streamlit, React, Next.js) -
Interface de chat - Histórico de conversas - Feedback (■) 2. API GATEWAY (Amazon API Gateway ou
Kong) - Autenticação (JWT, API Key) - Rate limiting (ex: 100 req/min por usuário) - SSL
termination 3. FASTAPI (Python) - Endpoints REST - Async para performance - Validação com Pydantic
- Retry logic para chamadas externas 4. VECTOR DATABASE (Qdrant, Weaviate, Pinecone) - Armazenar
embeddings dos documentos - Busca semântica (cosine similarity) - Filtros por metadata (ex:
filtrar por tipo de documento) 5. LLM (Bedrock / OpenAI API) - Geração de respostas - Structured
output para parsing - Fallback entre modelos 6. S3 (Object Storage) - Armazenar documentos
originais (PDFs, txt) - Versioning de documentos 7. CLOUDWATCH (Monitoring) - Métricas: latência,
custo, erros - Logs centralizados - Alarmes CÓDIGO – PIPELINE COMPLETO: import boto3 from fastapi
import FastAPI, HTTPException from pydantic import BaseModel app = FastAPI() class
ChatRequest(BaseModel): pergunta: str usuario_id: str class ChatResponse(BaseModel): resposta: str
fontes: list[str] latencia_ms: float @app.post('/chat', response_model=ChatResponse) async def
chat(request: ChatRequest): start = time.time() # 1. Embedding da pergunta embedding =
get_embedding(request.pergunta) # 2. Busca vetorial chunks = vectorstore.similarity_search(
query=request.pergunta, k=5, filter={'categoria': 'home_equity'}) # filtro por categoria ) # 3.
Monta contexto contexto = '\n'.join([f'[{i+1}] {c.page_content}' for i, c in enumerate(chunks)]) #
4. Chama LLM resposta = llm.invoke(f"""Contexto: {contexto} Pergunta: {request.pergunta} Responda
com base no contexto. Cite as fontes como [1], [2], etc.""") # 5. Log para CloudWatch
log_request(request.usuario_id, len(chunks), time.time() - start) return ChatResponse(
resposta=resposta.content, fontes=[c.metadata['source'] for c in chunks],
latencia_ms=(time.time() - start) * 1000 ) # ENDPOINTS: # POST /chat – chat principal com RAG #
GET /health – health check # POST /admin/reindex – reindexar documentos # GET /metrics – métricas
de uso # INFRA AS CODE (Terraform): resource "aws_lambda_function" "chatbot" { function_name =
"chatbot-bari" runtime = "python3.11" handler = "app.handler" memory_size = 512 timeout = 30
environment { variables = { QDRANT_URL = var.qdrant_url OPENAI_API_KEY = var.openai_api_key } } }
resource "aws_apigatewayv2_api" "chatbot_api" { name ="chatbot-bari-api" protocol_type = "HTTP" }
resource "aws_apigatewayv2_integration" "lambda" { api_id = aws_apigatewayv2_api.chatbot_api.id
integration_uri = aws_lambda_function.chatbot.invoke_arn integration_type = "AWS_PROXY" } resource
"aws_apigatewayv2_route" "chat" { api_id = aws_apigatewayv2_api.chatbot_api.id route_key = "POST

```

```
/chat" target = "integrations/${aws_apigatewayv2_integration.lambda.id}" }
```

System design para chatbot RAG envolve: frontend, API/backend, vector DB, LLM, monitoramento. Para júnior, não é esperado dimensionar infraestrutura, mas saber os componentes e como se conectam.

■ SEÇÃO 7 — COMPORTAMENTAIS E SITUACIONAIS

Esta seção é para perguntas de entrevista comportamental. O formato mais comum é STAR: Situação → Tarefa → Ação → Resultado. O ideal é usar exemplos reais do seu histórico. Se você não tem experiência profissional, use projetos pessoais, estágios, ou projetos de curso.

■ Me conte sobre um projeto de IA que você entregou do zero ao produção.

MODELO DE RESPOSTA STAR: SITUAÇÃO (1-2 frases): "Na minha última posição como estudante de desenvolvimento, criei um chatbot de IA para ajudar clientes de uma imobiliária a encontrar imóveis com base em preferências em linguagem natural." TAREFA (1-2 frases): "Eu precisava resolver o problema de clientes abandonando o site por não conseguirem encontrar imóveis via busca por filtros tradicionais." AÇÃO (3-5 frases, o foco): "1. Implementei um sistema RAG do zero usando LangChain + Chroma + GPT-3.5-turbo. 2. Criei um pipeline de ETL em Python para processar 500+ listings de imóveis. 3. Configurei chunking de 800 tokens com overlap de 100 tokens (melhorou precisão em 40%). 4. Integrei ao frontend via FastAPI com streaming de respostas. 5. Deployei tudo em streamlit cloud com CI/CD via GitHub Actions. 6. Implementei observabilidade com logging de latência e custos no CloudWatch." RESULTADO (2-3 frases, quantifique): "O chatbot reduziu o tempo médio de busca de 8 minutos para 45 segundos. O ticket médio de contatos aumentou 23% (de R\$ 280mil para R\$ 345mil por mês). O projeto foi aprovado como feature padrão do produto." DICA: Se você não tem projeto profissional, use projeto pessoal: - Participei de hackathons - Criei bot para meu TCC - Desenvolvi ferramenta interna para otimizar processo do trabalho O importante não é a escala, é demonstrar: 1. Entendimento técnico (escolhas certas) 2. Ownership (entregar do zero) 3. Resultado (conseguir medir impacto)

Esta é a pergunta mais comum em entrevistas de IA. Estruture com STAR e escolha um projeto que demonstre: capacidade técnica (RAG, APIs, embeddings), propriedade (você liderou), e resultado mensurável (mesmo que pequeno).

■ Como você explicaria RAG para alguém que não é técnico (ex: gerente de produto)?

MODELO DE RESPOSTA (simples e visual): "Imagine que você tem uma biblioteca enorme com todos os documentos do Banco Bari: contratos, políticas, manual de produtos. Um cliente pergunta: 'Posso usar meu apartamento de R\$ 500mil como garantia?' TRADICIONAL (sem RAG): O chatbot teria que 'decorar' todas as respostas possíveis durante o treinamento. Problemas: informação fica desatualizada, não sabecitei fontes, dá respostas erradas. COM RAG: 1. Cliente faz pergunta 2. O sistema busca os documentos mais relevantes sobre home equity 3. O chatbot lê esses trechos e responde com base neles 4. Se o documento muda, a resposta muda automaticamente – sem retreino Analogia simples: - Chatbot sem RAG = aluno que faz prova de memória (pode errar/fabricar) - Chatbot com RAG = aluno com livro aberto e pode consultar (resposta precisa) Benefícios para o negócio: ■ Respostas com fontes (cliente confia mais) ■ Informação atualizada (documento novo = resposta nova) ■ Custos menores (não precisa retreinar modelo) ■ Controle de qualidade (sabemos exatamente quais docs o chatbot usou) Limitação: ■■ Precisa de boa gestão de documentos (garbage in, garbage out)

Testa sua capacidade de comunicar conceitos técnicos complexos de forma simples. Gerentes de produto precisam entender para tomar decisões sobre priorização e roadmap.

■ Você recebe feedback que o chatbot está dando respostas ruins. O que faz?

FLUXO DE DEBUG DE CHATBOT: PASSO 1: DIAGNOSTICAR (não corrigir ainda) - Verificar logs: quais perguntas estão falhando? - Classificar erros por tipo: - Retrieval falhou (buscou contexto errado) - Generation falhou (modelo entendeu mal) - Contexto insuficiente (documento não cobre tema) PASSO 2: MÉTRICAS - Precision@5: dos 5 docs retrievados, quantos são relevantes? - Faithfulness: a resposta segue o contexto retrievado? - Answer relevance: a resposta responde a

pergunta? - Citar fontes: % de respostas com citations PASSO 3: CAUSA RAIZ Cenário A: Retrieval falhou → Verificar qualidade dos embeddings → Testar chunking (chunks muito grandes ou pequenos?) → Verificar se documentos relevantes estão no banco → Testar search com query semelhante manualmente Cenário B: Generation falhou → Prompt não está instruindo corretamente → Contexto retrievado não é suficiente (poucos docs) → Temperature muito alto (alucinação) Cenário C: Contexto insuficiente → Documento sobre o tema não existe → criar → Metadados errados (filtro exclui docs relevantes) → Query expansion não está funcionando PASSO 4: CORREÇÕES ITERATIVAS 1. AJUSTAR CHUNKING - Reduzir chunk size de 1000 para 500 tokens - Aumentar overlap de 50 para 100 tokens - Testar chunking semântico (por parágrafo) 2. OTIMIZAR RETRIEVAL - Testar diferentes embedding models - Adicionar bm25 (hybrid search) junto com vector search - Implementar re-ranker (cross-encoder) 3. AJUSTAR PROMPT - Adicionar instruction: 'Se não sabe, diga que não sabe' - Adicionar exemplos few-shot - Especificar formato de resposta (bullets, citações) 4. ADICIONAR MONITORAMENTO - A/B test: nova versão vs versão atual - Feature flags para ativar/desativar mudanças PASSO 5: VALIDAR - Comparar métricas antes/depois - Teste com queries reais de usuários - Verificar se erros não estão mais acontecendo

Este é um problema real de produção. A resposta deve mostrar método: diagnosticar antes de corrigir, usar

Este é um problema real de produção. A resposta deve mostrar método: diagnosticar antes de corrigir, usar métricas, iterar sistematicamente.

■ Como você prioriza quando tem múltiplas demandas de IA ao mesmo tempo?

MATRIZ DE PRIORIZAÇÃO (Impacto x Esforço): ESFORÇO Baixo Alto

Alto ■ FAZER ■ PLANEJAR ■ Impacto ■ AGORA ■ (Q2/Q3) ■
Baixo ■ DELEGAR ■ IGNORAR ■ ou batch ■ ou drop ■

[illegible]

de home equity (alta, 20h esforço) → FAZER AGORA: alto impacto + viabilidade alta Demanda 2: Análise de crédito com ML (muito alto impacto, 80h) → PLANEJAR: alto impacto mas precisa de mais tempo/recursos Demanda 3: Relatório de NPS automatizado (baixo impacto, 5h) → DELEGAR: automação simples, não sou eu que preciso fazer Demanda 4: Detecção de fraude em tempo real (muito alto, 200h) → PLANEJAR: projeto complexo, requer arquitetura específica

NEGOCIAÇÃO COM SOLICITANTES: Quando uma demanda é alta urgência mas baixa importância: - "Posso fazer, mas preciso garantir que a demanda X (mais estratégica) não seja afetada. Podemos discutir um prazo?" Quando há conflito de prioridades entre áreas: - Levar para gestor com dados: "A área A pede feature X (impacto R\$ Y), área B pede feature Z (impacto R\$ W). Sugiro priorizar X baseado em ROI"

FRAMEWORK DE DECISÃO: 1. Impacto no negócio (receita, redução de custo, eficiência) 2. Viabilidade técnica (tempo, complexidade, riscos) 3. Precedência estratégica (alinhamento com objetivos do Bari) 4. Dependências (bloqueia outras entregas?)

COMUNICAÇÃO: - Sempre informar o status ao solicitante - Se não posso entregar no prazo, avisar ANTES, não DEPOIS - Sugerir alternativas: 'Posso entregar 80% do valor em 50% do tempo'

Tanto a gestão de prioridades e a capacidade da negociação. A resposta deve mostrar framework de decisão, não

Esta gestão de prioridades e capacidade de negociação. A resposta deve mostrar framework de decisão, não apenas 'faço tudo' (que é **errada**).

■ SEÇÃO 8 — TERMOS TÉCNICOS EM PORTUGUÊS

Entrevistas no Brasil podem misturar termos em inglês e português. Esta seção lista os termos mais comuns para você estar preparado.

- Backend / Backend** — Lógica do servidor, API, banco de dados
- Frontend** — Interface que o usuário vê (HTML, CSS, JS, React)
- Deploy** — Publicar código em ambiente de produção
- Stack** — Conjunto de tecnologias (Python + FastAPI + Qdrant)
- Bug** — Erro no código que causa comportamento inesperado
- Debug** — Processo de encontrar e corrigir bugs
- Framework** — Estrutura que facilita desenvolvimento (FastAPI, LangChain)
- Library / Lib** — Biblioteca de código reutilizável
- API** — Interface de comunicação entre sistemas
- Query** — Consulta a banco de dados ou busca
- Endpoint** — URL específica de uma API (/chat, /health)
- Token** — Unidade de texto para LLMs; também credencial de autenticação
- Fine-tuning** — Treinar modelo com dados customizados
- Inference** — Processo de gerar resposta com modelo
- Streaming** — Resposta que chega aos poucos (não espera tudo)
- Middleware** — Código que executa entre requisição e resposta
- CI/CD** — Integração contínua / deploy contínuo
- Mock** — Objeto simulando outro para testes
- Refatorar** — Reescrever código mantendo funcionalidade
- Review** — Revisão de código por outro desenvolvedor
- PR (Pull Request)** — Solicitação para incorporar código na main
- Branch** — Versão paralela do código para desenvolver feature
- Merge** — Unir branch de volta na main
- Feature** — Nova funcionalidade
- Tech debt** — Dívida técnica — código que precisa ser refatorado
- Sprint** — Ciclo de desenvolvimento (geralmente 2 semanas)
- MVP** — Produto mínimo viável — versão mais simples que funciona
- Webhook** — Notificação via HTTP quando algo acontece
- Schema** — Estrutura de dados ou banco
- Index** — Estrutura para acelerar buscas em banco

Boa sorte na entrevista! Lembre-se: o mais importante é demonstrar capacidade de raciocínio lógico, vontade de aprender, e comunicação clara — mais do que saber tudo de memória. Se não souber algo, diga 'não sei, mas sei onde buscar'.